

腾讯云 AI Skills 零门槛上手！立即体验

文档 建议反馈 控制台

首页 学习 活动 专区 圈层 工具 MCP广场

AI 大模型 会用再买，即刻体验

文章/答案/技术大牛

社区首页 > 专栏 > Harness Engineering 最佳实践：长运行多智能体的框架设计

Harness Engineering 最佳实践：长运行多智能体的框架设计

修改于 2026-04-03 19:53:38

6.8K

8

举报

概述

本文基于 Anthropic 工程团队关于 Harness Design 的理论框架，结合 Harness Engineering —— 一个基于 Claude Code 的多智能体编排桌面应用的完整实践经验，探讨如何设计、实现和优化长时运行的 AI 开发流水线。

一、为什么需要 Harness?

1.1 单智能体的天花板

当我们第一次把一个复杂需求丢给单个 Claude 实例时，结果往往令人失望。给它 20 分钟、花费 \$9，它也许能生成一个“看起来像那么回事”的骨架。但仔细检查你会发现：游戏逻辑不能运行，API 接口互相矛盾，安全漏洞随处可见。这不是模型能力的问题，而是认知架构的问题。

Anthropic 工程团队的研究指出了两个持久的失败模式：

上下文窗口的侵蚀：随着对话历史不断膨胀，模型对早期决策的记忆开始模糊。它在第 50 轮对话中做出的选择，可能与第 5 轮的架构决策完全矛盾。我们在实践中观察到，当一个 Generator Agent 的执行超过 40 轮工具调用时，代码风格的一致性会显著下降。

自我评估的失灵：模型天然倾向于对自己的输出给予正面评价。当你让同一个 Claude 实例先写代码再审查代码，它几乎总是说“代码质量良好”。这不是幻觉，而是一种更深层的认知偏差——它很难对自己刚刚做出的决策保持批判距离。

Harness 正是为了突破这两个瓶颈而存在。它不是一个 Prompt 技巧，而是一种系统架构。

1.2 从 GAN 到 Multi-Agent Pipeline

Anthropic 的核心洞见来自一个优雅的类比：生成对抗网络（GAN）。在 GAN 中，Generator 和 Discriminator 通过对抗训练互相提升。将这个思想迁移到 AI 编程领域：Generator 负责写代码，Evaluator 负责判断代码质量。

“将做工作的智能体与评判工作的智能体分离，是一个强有力的杠杆”——因为让评估者变得更加怀疑，远比让生成者变得更加自我批判要容易得多。 — Anthropic Engineering

在 Harness Engineering 中，我们将这个二元对立推广为一个六智能体流水线：

代码语言：TXT



自动换行



AI代码解释

用户需求 → Planner → Generator → Code Reviewer → Security Reviewer → QA Engineer → 交付

每个智能体都有独立的认知边界和评判标准，它们之间的“张力”构成了整个系统的质量保障。

二、智能体团队架构设计

童子龙

LV.2

挚文集团（陌陌&探探...
基础平

文章 7

作者相关精选

IDC Redis如何基于云DTS实

HDFS元信息管理的核心技术

性能提升30%，陌陌应用性

目录

一、为什么需要 Harness?

1.1 单智能体的天花板

1.2 从 GAN 到 Multi-Agent Pip

二、智能体团队架构设计

2.1 智能体角色定义

几个值得深思的设计决策

2.2 流水线编排逻辑

2.3 评估体系设计与调优

交个朋友

加入腾讯云官网粉丝团

蹲全网底价单品 享第一手活动
信息



相关产品与服务

云服务器

云服务器（Cloud Virtual Machine，
靠的弹性计算服务。您可以实时扩展

产品介绍

产品文档

4核4G3M云服务器 新用户

2.1 智能体角色定义

角色	核心职责	推荐模型	工具权限	设计要点
Planner（规划器）	理解需求、拆解任务、定义依赖和完成标准	opus	Read <u>Glob</u> Grep	输出结构化任务卡，作为后续流水线输入
Generator（生成器）	实现代码、补测试、完成任务	sonnet	Read <u>Write</u> Edit <u>Bash</u> Glob / Grep	主力开发者，负责把任务落成代码
Code Reviewer（代码质量审查员）	审查代码质量、边界处理、架构合理性和测试有效性	opus	Read <u>Bash</u> Glob / Grep	只审不改，输出 VE RDICT: PASS / FAIL
Security Reviewer（安全审计员）	审查注入、认证授权、敏感信息泄露和依赖安全	opus	Read <u>Bash</u> Glob / Grep	独立安全门禁，避免安全问题被功能问题掩盖
QA Engineer（质量保证工程师）	执行测试、验证回归、检查构建和验收结果	sonnet	Read <u>Bash</u> Glob / Grep	负责“跑代码”，验证是否真的可交付
Debugger（系统化调试器）	根据失败报告定位问题并做最小化修复	sonnet	Read <u>Write</u> Edit <u>Bash</u> Glob / Grep	仅在失败路径激活，修完后重新进入评审和 QA

- **Planner：**
适合高推理模型，因为它决定任务拆解质量。实践上应强制输出 JSON 或其他结构化格式，方便直接生成 Sprint 卡片。
- **Generator：**
更适合高吞吐模型，因为实现阶段通常伴随大量读写文件、执行命令和反复迭代。Prompt 应明确要求“优先产出可验证结果”，而不是只生成表面完整的代码。
- **Code Reviewer：**
关键是保持独立性。不给写权限，才能避免它从“审查者”滑向“修复者”，plan使用claude模型，cr使用gpt模型。
- **Security Reviewer：**
建议与 Code Reviewer 并行执行，各自独立给出结论。这样即使功能评审通过，安全问题仍然可以单独拦截。
- **QA Engineer：**
不是再读一遍代码，而是通过测试、构建和回归验证确认任务是否真的满足完成标准。
- **Debugger：**
只处理失败报告里明确指出的问题，遵循最小化修复原则，避免“修一个 Bug 顺手重构半个项目”。

几个值得深思的设计决策

模型选择的差异化：Planner 和 Reviewer 使用 opus（更强的推理能力），Generator 和 QA 使用 sonnet（更快的执行速度）。规划和审查需要深度思考，而代码生成和测试执行需要高效的工具调用吞吐量。

工具集的权限隔离：Code Reviewer 和 Security Reviewer 没有 Write 和 Edit 权限。评审者只能阅读和分析，不能修改代码。这确保了评审的独立性——如果评审者可以直接修改代码，它就会倾向于"修复后通过"而不是"标记问题并拒绝"。

轮次预算的约束：每个智能体都有硬性的轮次上限。这不仅是成本控制，更是一种认知卫生：它迫使每个智能体在有限的"注意力预算"内完成工作，防止无止境的自我修正循环。

2.2 流水线编排逻辑

Pipeline 的核心执行流程体现了"分治-评估-迭代"的哲学。几个核心机制值得关注：

文件快照 Diff 机制：在每个任务开始前，系统拍摄项目目录的完整快照（文件路径 + 大小 + 修改时间）。任务完成后再拍一次快照，通过 Diff 精确识别出哪些文件是本次生成/修改的。这解决了一个实际问题——当多个任务在同一个项目目录中执行时，你需要知道每个 Generator 到底改了什么。

VERDICT 协议：Reviewer 和 QA 的输出必须以 **VERDICT: PASS** 或 **VERDICT: FAIL** 结尾。这将主观的“代码看起来还行”转化为二元的通过/失败信号，使得流水线可以基于这个信号做出自动化决策。

优雅降级策略：当 Reviewer 失败时，Pipeline 不会中断——它记录错误并继续下一步。当 QA 失败时，任务会被标记为 qa-failed 并回退到 review 状态。这种分级的容错设计确保了长时运行的 Pipeline 不会因为单点故障而完全崩溃。

2.3 评估体系设计与调优

架构层面的智能体分工和流水线编排只是 Harness 的“骨架”，而评估体系的质量才是“灵魂”。Anthropic 在实践中发现，大多数 Harness 的产出质量问题不是出在生成器上，而是出在评估器上——一个宽松的评估器会让整条流水线失去意义。以下三个维度是构建高质量评估体系的关键。

维度一：结构化评分框架

Anthropic 在前端设计评估场景中提出了四维评分体系，将「好不好」拆解为可独立打分的维度。这个方法论直接适用于代码评审和 QA 验收：

- **Design Quality（设计质量 / 风格一致性）：**模块间的命名规范、错误处理模式、API 设计风格是否全局统一。
- **Originality（原创性）：**是否有自定义的设计决策，而非套用模板。Anthropic 特别提到要惩罚「AI slop」——千篇一律的模板代码输出。
- **Craft（工艺性）：**边界条件覆盖、类型安全、性能热点处理、日志充分等技术执行的精细度。
- **Functionality（功能性）：**功能是否按预期工作、测试是否通过、用户路径是否畅通——独立于代码美学的可用性判断。

关键实践：在 Code Reviewer 和 Security Reviewer 的 Prompt 中提供 few-shot 样本校准——即带有详细分数分解的示例，明确定义 PASS 和 FAIL 的边界线。这可以有效防止「分数漂移」（评估器随时间推移逐渐降低或抬高评分标准）。

维度二：QA 反馈的具体性原则

Anthropic 强调，优秀的评估器不是输出「代码质量良好」，而是输出可执行的 Bug 报告。每个 **VERDICT: FAIL** 必须包含三要素：

- **在哪里：**文件路径、函数名、行号。不说「拖拽功能有问题」，而说「Tool 只在拖拽起点和终点放置瓷砖，未填充中间区域」。
- **什么现象：**实际行为 vs 预期行为。不说「API 报错」，而说「FastAPI 将 reorder 字符串匹配为 frame_id 的整数参数，返回 422」。
- **为什么：**根因分析。这样 Debugger 才能快速定位问题，而不是在 60 轮预算中浪费时间重复发现同一个 Bug。

维度三：评估器的迭代调优方法论

Anthropic 披露了一个被很多实践者忽视的现实：初始版本的评估器几乎总是失败的。真正有效的评估器是「调」出来的，不是「写」出来的。调优循环分四步：

1. **系统性阅读评估器日志：**不只看 PASS/FAIL，而是通过 Agent Teams 的 Thinking Tab 逐行阅读评估器的推理过程，关注它「看到了什么」「忽略了什么」「给了什么权重」。
2. **识别判断偏差：**将评估器判断与人类判断做对比。「过于宽松」（该 FAIL 时 PASS）和「过于严苛」（对无关紧要的问题 FAIL）需要不同的 Prompt 修正策略。
3. **更新 Prompt 并复测：**用同一批测试用例复测，通常需 3-5 轮迭代才能将评估器的判断对齐到可接受标准。

4. **持续监控漂移**：定期人工抽检评估结果，尤其是更换模型版本后。Live Feed 和 Agent Teams 页面为这个调优循环提供了基础设施。

三、Stream-JSON 与实时可观测性

Anthropic 的文章提到一个关键点：长时运行的任务不透明是最大的风险。如果你启动一个 6 小时、\$200 成本的 Pipeline，却只能在最后看到结果，那么中途的任何偏差都会被放大为灾难性的浪费。

Harness Engineering 的解决方案是全链路实时流式可观测性。Claude Code CLI 支持 `--output-format stream-json` 模式，将每个工具调用、思考过程、文本输出都作为独立的 JSON 事件流式输出。我们的 `processStreamEvent` 函数实时解析这些事件，并通过 `classifyTool()` 对每个工具调用进行 MCP/Skill/Plugin/Built-in 分类。

这些信息驱动了一个关键的 UI 功能——**Skills & MCP Tab**。当你在 Agent Teams 页面选中一个正在运行的 Agent，你可以实时看到：

- 这个 Agent 调用了哪些 MCP 服务器，每个服务器下使用了哪些具体工具
- 触发了哪些 Skill，每个 Skill 被调用了几次以及参数摘要
- Built-in 工具（Read、Write、Bash 等）的调用频次分布

这些信息对于调试和优化 Agent 行为至关重要。如果你发现 Generator 在某个任务上疯狂调用 Bash 而几乎不用 Write，这可能意味着它在试图通过脚本生成代码而不是直接编写——这通常是一个需要干预的信号。

四、上下文管理：Harness 最难的问题

4.1 Context Reset vs. Context Continuation

Anthropic 的文章指出了一个深刻的权衡："与在单一会话中通过摘要来保持连续性不同，上下文重置提供了干净的开始，消除了'上下文焦虑'——但代价是需要结构化的交接产物。"

在 Harness Engineering 中，我们选择了 context reset 策略：每个智能体的每次执行都是一个全新的 Claude Code 进程。Planner 的输出不会作为 Generator 的对话历史传递，而是被结构化为 JSON 任务描述后作为 Generator 的 Prompt 的一部分。这消除了上下文污染——Generator 不会被 Planner 的推理过程所影响，它只关注最终的任务定义。

4.2 Sprint 合约：可验证的交接

Anthropic 工程团队提出了 Sprint Contract 的概念——"在编码开始之前，关于完成标准的显式协议"。在我们的实现中，Sprint Board 上的每张卡片就是一个 Sprint 合约。当 Planner 创建任务、Generator 认领执行、Reviewer 审查通过、QA 验收完成——这张卡片的状态变化（inbox → in_progress → review → done）就是一条可追溯的质量链。

4.3 Prompt 工程与模型脚手架的可退化性

```
## "Harness 中的每个组件都编码了一个关于模型不能独立完成什么的假设，而这些假设需要随着模型的改进而定期重新评估。" — Anthropic Engineering ##
```

Anthropic 发现，Opus 4.6 在规划、上下文管理和调试方面的改进，减少了对 Sprint 分解的需求。这意味着曾经必不可少的 Harness 组件，可能在模型升级后变成了不必要的开销。对我们的启示是：好的 Harness 设计应该是可退化的——当模型足够强大时，你应该能够关闭某些中间环节，而不需要重写整个流水线。

Prompt 语言的引导效应

与模型能力演进同样重要的是 Prompt 措辞的影响。Anthropic 发现，Prompt 中的表述会深刻塑造输出的「性格」——强调「production-grade」会让 Generator 增加防御性代码；强调「rapid prototype」会让它跳过最佳实践。这些措辞不是修饰语，而是调控智能体行为边界的核心参数。具体建议：



- **Planner** → 「ambitious scope」：避免功能规划不足，主动提议更完整的方案。
- **Code Reviewer** → 「critical and skeptical」：抵消模型天然的宽大倾向，明确要求「对每个函数都假设它有问题」。
- **Generator** → 「incremental and testable」：确保每步产出都可验证，而非一次性生成整个工程。

更重要的发现：改进不是线性的。后一轮迭代的输出不一定比前一轮好。因此应保留每轮迭代的产物（Agent Teams 页面保留最近 20 次执行历史），允许人工回溯和选择最佳版本。

五、状态持久化：长时任务的生命线

一个 6 小时的 Pipeline 运行到第 5 小时时进程崩溃了——这在真实环境中一定会发生。如果没有状态持久化，5 小时的工作和 \$150+ 的成本就全部归零。

Harness Engineering 的持久化策略分为三层：

- **实时关键事件保存**：每个 Agent 的执行历史（thinking、tools、output）在执行完成的瞬间就被写入磁盘。即使下一秒进程崩溃，这次执行的完整记录已经安全了。
- **定时自动保存**：每 30 秒自动保存一次，确保即使 Agent 正在执行中，最多丢失半分钟的数据。
- **退出前保存**：无论是用户主动关闭窗口，还是系统强制退出，都会触发最后一次保存。

保存的内容包括：Sprint Board 上的所有任务、每个 Agent 的执行历史（最近 20 次）、最近 200 条事件日志、Chat 对话记录。重启后，持久化的 Chat 消息通过专用 channel 恢复，避免与实时消息混淆。

一个微妙的设计决策：正在运行的执行（current）不会被保存，因为重启后进程已经不存在了。与其保存一个不完整的执行状态，不如干净地标记"上次执行被中断"。

六、插件与技能生态：超越内置能力

Harness Engineering 不仅编排 Agent，还深度集成了 Claude Code 的插件生态系统。通过扫描 `~/.claude/plugins/installed_plugins.json`，我们可以发现所有已安装的插件及其提供的 Skills、MCP 服务器和 Commands。Skills Hub 和 MCP Hub 不是静态的推荐页面——它们是 Claude Code 实际安装环境的实时镜像。

一个关键的实践经验：`~/.claude.json` 文件不能被整体当作 MCP 配置读取。这个文件包含了大量 Claude Code 的内部状态（TipsHistory、CachedStatsigGates 等），如果使用错误的 fallback 逻辑，会把这些内部字段误识别为 MCP 服务器。正确的做法是严格只读 `mcpServers` 字段，并验证每个条目都有 command 或 url 属性。

一个独立的多智能体编排器只是一个工具。但当它与 Claude Code 的 Skills 和 MCP 生态深度集成后，它变成了一个平台。在 Harness Engineering 中，162 个 Skills 和 18 个 MCP 服务器是每个 Agent 可以动态调用的能力。这种动态的能力发现和调用机制，使得 Harness 是一个持续进化的开发智能体团队。

七、内生式 AI 功能集成：Endogenous AI Assistant 模式

Anthropic 提出了一个精妙的实践：在 Planner 的 Prompt 中明确要求「将 AI 功能织入产品规格」。结果是，生成的应用内部包含了 Claude 集成的生成式功能——这些不是简单的 API 调用封装，而是「真正的 Agent」，能够通过工具调用驱动应用自身的功能。这形成了一个有趣的递归结构：AI 智能体生成包含 AI 智能体的应用。

在 Harness Engineering 中，这意味着插件生态不仅服务于「生成过程」，也应该服务于「生成结果」。Planner 的 Prompt 可以增加指导：「主动识别可以通过 AI 增强的功能点，而不仅仅是生成静态代码」。当 Skills Hub 中的 162 个 Skill 和 MCP Hub 中的 18 个 Server 不仅被智能体在开发过程中使用，还被织入生成产物本身时，插件生态的价值就实现了乘数级放大。

八、反思：Harness 设计的哲学



8.1 每个组件都是一个假设

回到 Anthropic 的核心洞见："Harness 中的每个组件都编码了一个关于模型不能独立完成什么的假设。"在 Harness Engineering 中：

- Planner 的存在假设模型不能在一次性交互中同时规划和实现一个复杂需求
- 独立 Reviewer 的存在假设模型不能对自己的代码保持批判距离
- VERDICT 协议假设模型不能自发地给出结构化的通过/失败判断
- 文件快照 Diff 假设模型不能可靠地自我报告它修改了哪些文件
- 30 轮次上限假设模型在超过一定对话长度后性能会下降

这些假设中，有些可能在下一代模型中不再成立。当那一天到来时，好的 Harness 设计应该允许你移除这些组件——而不需要重新设计整个系统。

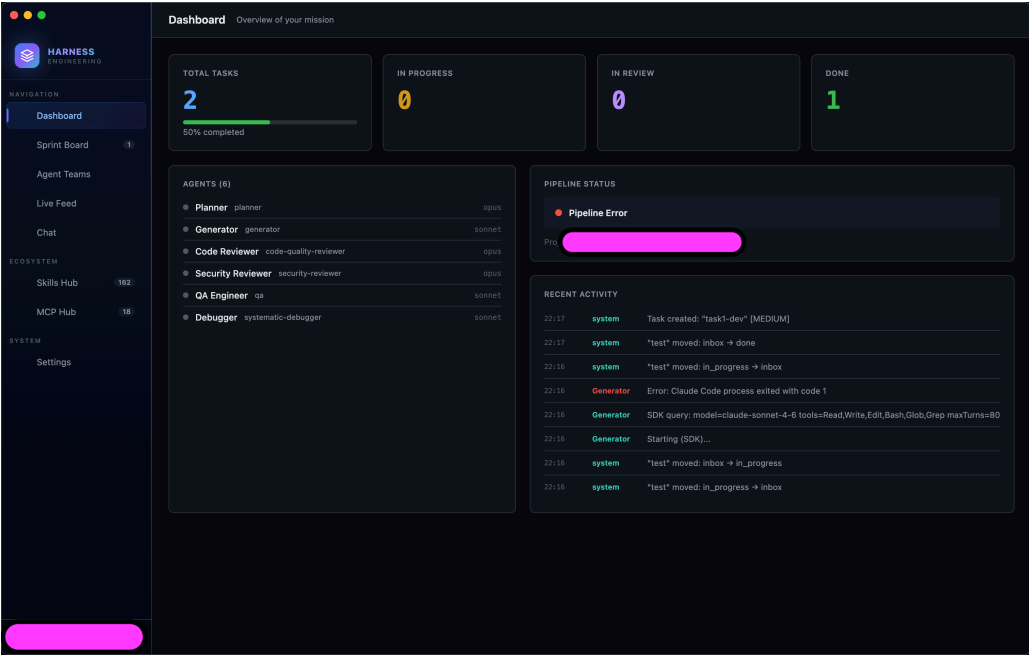
8.2 可观测性 > 自动化

一个违反直觉的经验：在 Harness 设计中，可观测性比自动化更重要。一个完全自动化但不透明的 Pipeline（你按下按钮，6 小时后得到结果），不如一个半自动但高度可观测的 Pipeline（你可以实时看到每个 Agent 在做什么、调用了哪些工具、产生了什么输出）。因为后者允许你在任何节点介入。这种"人在环中"（Human-in-the-Loop）的能力，在长时运行任务中的价值是无可替代的。

九、Harness 工程落地实践全景

理论需要实践来验证。以下是作者日均消费数亿claude 4.6 ops token的实践经验，最后落地成一套自研的AI Native研发全流程的桌面端，各核心模块的实战解析，每个模块都从"做什么""怎么做""为什么这么做"三个维度展开，将前文的设计哲学落地为可视化的产品形态。

9.1 Dashboard — 任务总览与指挥中心



Dashboard 是整个 Harness Engineering 的指挥中心，在一个页面内聚合了任务进度、智能体状态、流水线运行和最近活动四大信息维度，让团队在 10 秒内掌握当前 Sprint 的全貌。

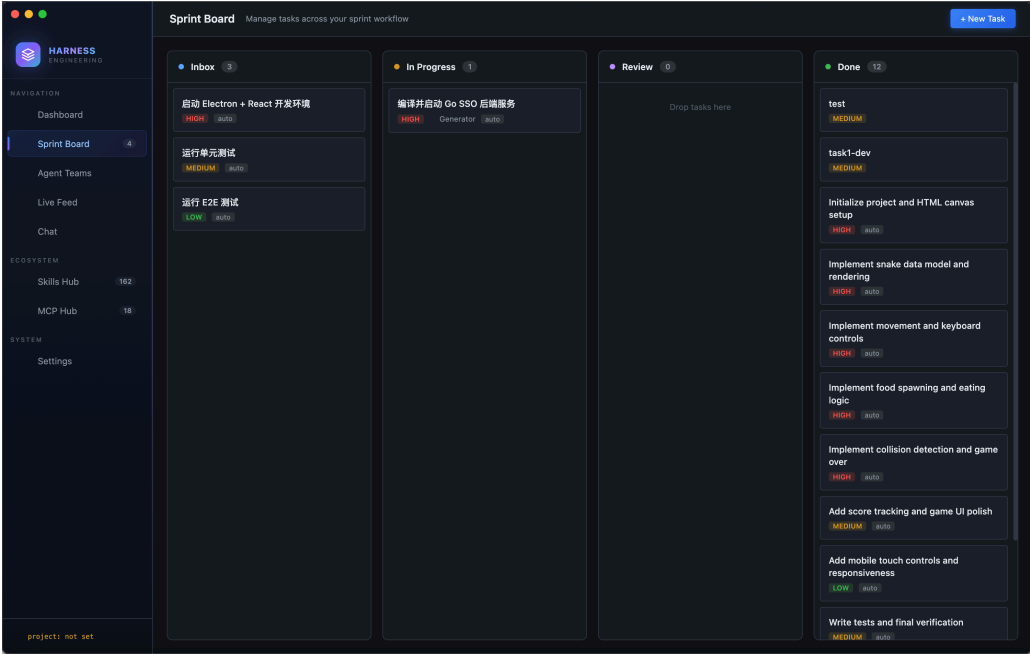
核心能力：

- 四维指标卡片：TOTAL TASKS、IN PROGRESS、IN REVIEW、DONE，配合进度条，实时反映当前迭代的健康度。当 IN REVIEW 堆积而 IN PROGRESS 为 0 时，说明流水线已完成生成但质量门禁拦截了大量任务。
- 智能体阵容面板：左侧 AGENTS (6) 区域展示完整团队阵容——Planner (opus)、Generator (sonnet)、Code Reviewer (opus)、Security Reviewer (opus)、QA Engineer (sonnet)、Debugger (sonnet)。每个智

- 能体均标注角色名称、模型选型和状态指示灯。
- **Pipeline 状态监控**：右侧显示当前流水线状态（Pipeline Idle Running_Complete）和工作目录路径，一笔确认当前的目标仓库。
 - **最近活动时间线**：RECENT ACTIVITY 以时间倒序展示关键事件：pipeline complete、QA Engineer 验证结果、Bash 命令调用等，相当于整个系统的"最近动态"消息流。

「模型选择差异化」原则——Reviewer 类智能体使用 opus（更强的推理能力），Generator 类使用 sonnet（更快的生成速度），而非全员使用同一模型。这是成本-质量权衡的具体落地：让右脑（生成）跑得快，让左脑（审查）想得深。

9.2 Sprint Board — 看板式任务编排与质量门禁



Sprint Board 是 Anthropic 提出的 Sprint Contract 理念的可视化实现。它将 Planner 拆解出的子任务按状态分布在四列看板上，让每个任务的生命周期一目了然，所有的任务会经过 meta-plan，AI 会自动判断任务的复杂度，来动态规划后续的执行 agent 流程。

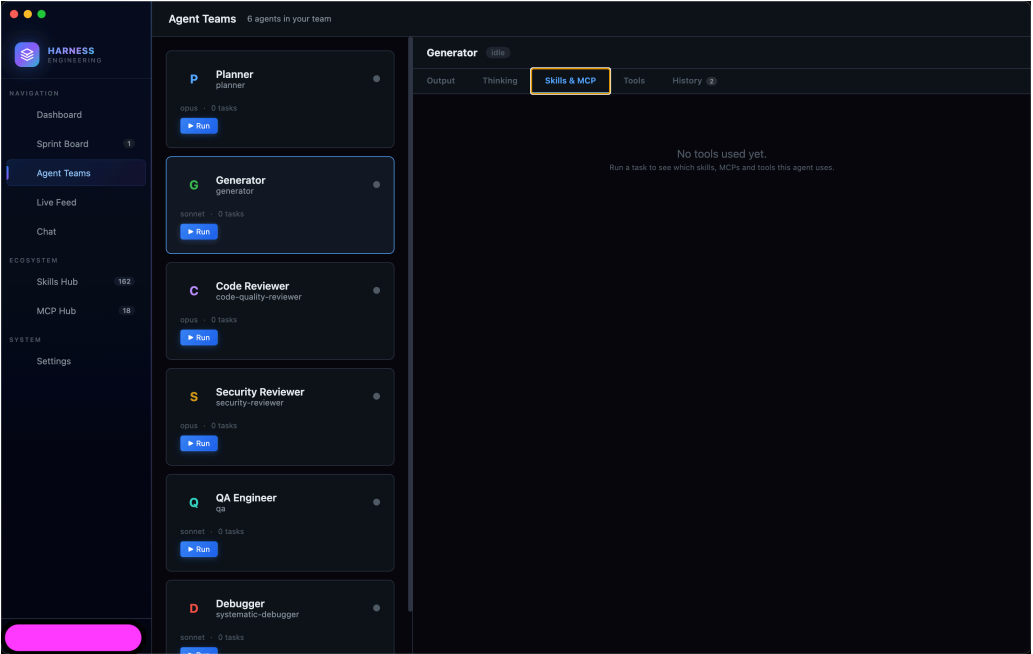
核心能力：

- **四列状态流转**：Inbox（待处理）→ In Progress（执行中）→ Review（审查中）→ Done（已完成）。任务卡片随 Pipeline 执行自动流转，无需人工拖拽。
- **三级标签体系**：每张任务卡片携带三类元信息——优先级标签（HIGH 红色 / MEDIUM 黄色）表明任务紧急度；来源标签（auto）表明由 Planner 自动拆解而非人工创建；QA 状态标签（qa-failed）表明未通过质量门禁。
- **智能拆解可视化**：Planner 将一个原始需求拆解为 7 个子任务（Planner会根据任务的复杂程度，动态规划 agent 执行流程）：Locate local repo clone、Check uncommitted changes via git status/diff、Review code changes for bugs、Check for security vulnerabilities、Verify tests still pass、Fix identified bugs if any、Stage and commit changes。拆解粒度精确到可执行操作级别。
- **质量门禁可视化**：6 个任务停留在 Review 列并标记 qa-failed，仅 1 个进入 Done。这不是 Bug，而是 V ERDICT 协议在发挥作用——QA Engineer 未给出 **VERDICT: PASS** 的任务不允许进入 Done。

这个看板是 Sprint 合约的可视化表达。Anthropic 提出："在编码开始之前，关于什么会被构建的共享理解"。Sprint Board 将这个"共享理解"变成了卡片、状态和标签。更关键的是，qa-failed 标签证明了第二章 VERDICT 协议的实效：当 6/7 任务被拦截在 Review 列时，系统宁可报告"只完成 1/7"也不会让低质量结果流入 Done。这正是「约束产生质量」的核心体现。

9.3 Agent Teams — 智能体团队管理与工具链追踪





Agent Teams 是多智能体协作的核心管控界面，每个 agent 都是一个具有专业 skills 的工程专家，通过对Claude code agent SDK的封装，它将六个智能体以卡片形式展示在左侧，点击任意卡片后右侧详情面板提供五维度透视：Output（最终输出）、Thinking（思考过程）、Skills & MCP（工具链）、Tools（原始调用）、History（历史记录）。

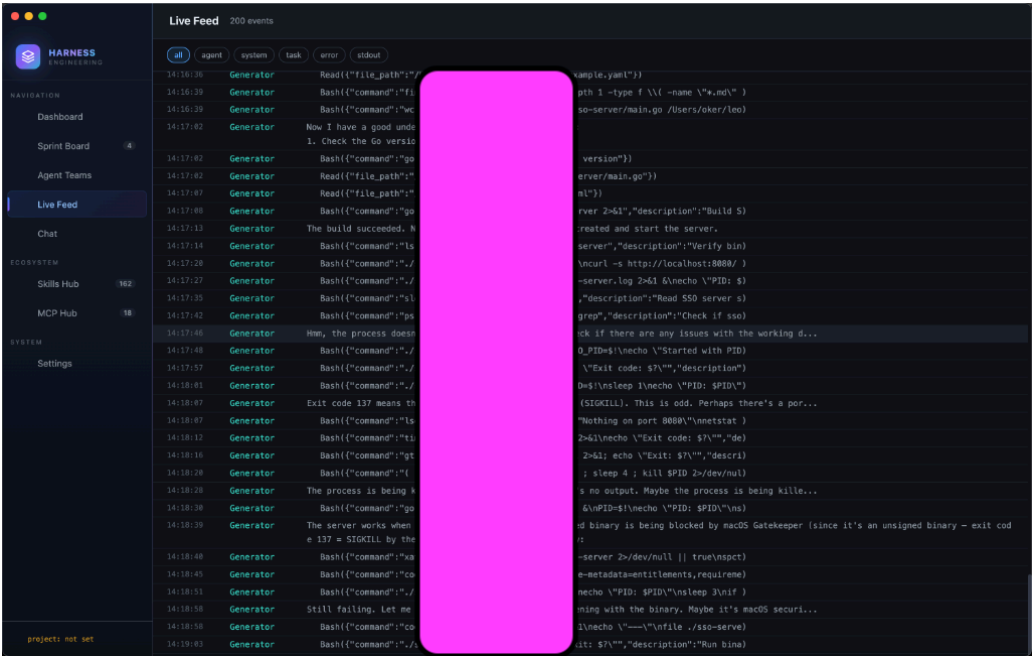
核心能力：

- 智能体卡片：每张卡片展示名称、角色标识（如 planner、generator、code-quality-reviewer）、模型选型（opus/sonnet）、已完成任务数、状态指示灯（灰色=idle，绿色=running，红色=error），以及独立的 Run 按钮支持单智能体调试。
- Skills & MCP Tab（重点功能）：通过解析 Claude Code 的 stream-json tool_use 事件，自动将智能体调用的工具分三类展示：① MCP Server 调用（如 mcp_github_create_issue，解析出 Server 名称和具体 Tool）；② Skills 调用（如 Skill → tdd、Skill → frontend-design，从 tool_use 的 input.skill 字段提取实际名称）；③ Built-in Tools（如 Read、Write、Bash、Grep，统计调用频次）。
- 多维度详情：Output Tab 显示智能体的最终输出（如 Review Summary、VERDICT: PASS/FAIL）；Thinking Tab 展示 Claude 的内部推理过程；Tools Tab 列出原始 tool_use 事件序列；History Tab 保留最近 20 次执行记录。

Skills & MCP Tab 是「Stream-JSON 与实时可观测性」的核心产品化实现。文章中提到「如果你发现 Generator 在某个任务上狂调 Bash 却不用 Read，可能意味着它的工具选择策略需要调优」，这正是 Skills & MCP Tab 解决的问题。它给每个智能体的工具调用装上了"计量器"，让优化从"感觉"变成"数据驱动"。每个智能体卡片上的 Run 按钮则体现了「每个组件都是假设」的哲学，允许单独触发任何智能体，便于快速验证假设是否成立。

9.4 Live Feed — 全链路实时事件流





Live Feed 是整个系统的“飞行记录仪”（Flight Recorder），实时捕捉并展示 Pipeline 执行过程中的每一个事件。它是 Anthropic 强调的「长时任务不透明是最大风险」这一判断的直接解决方案。

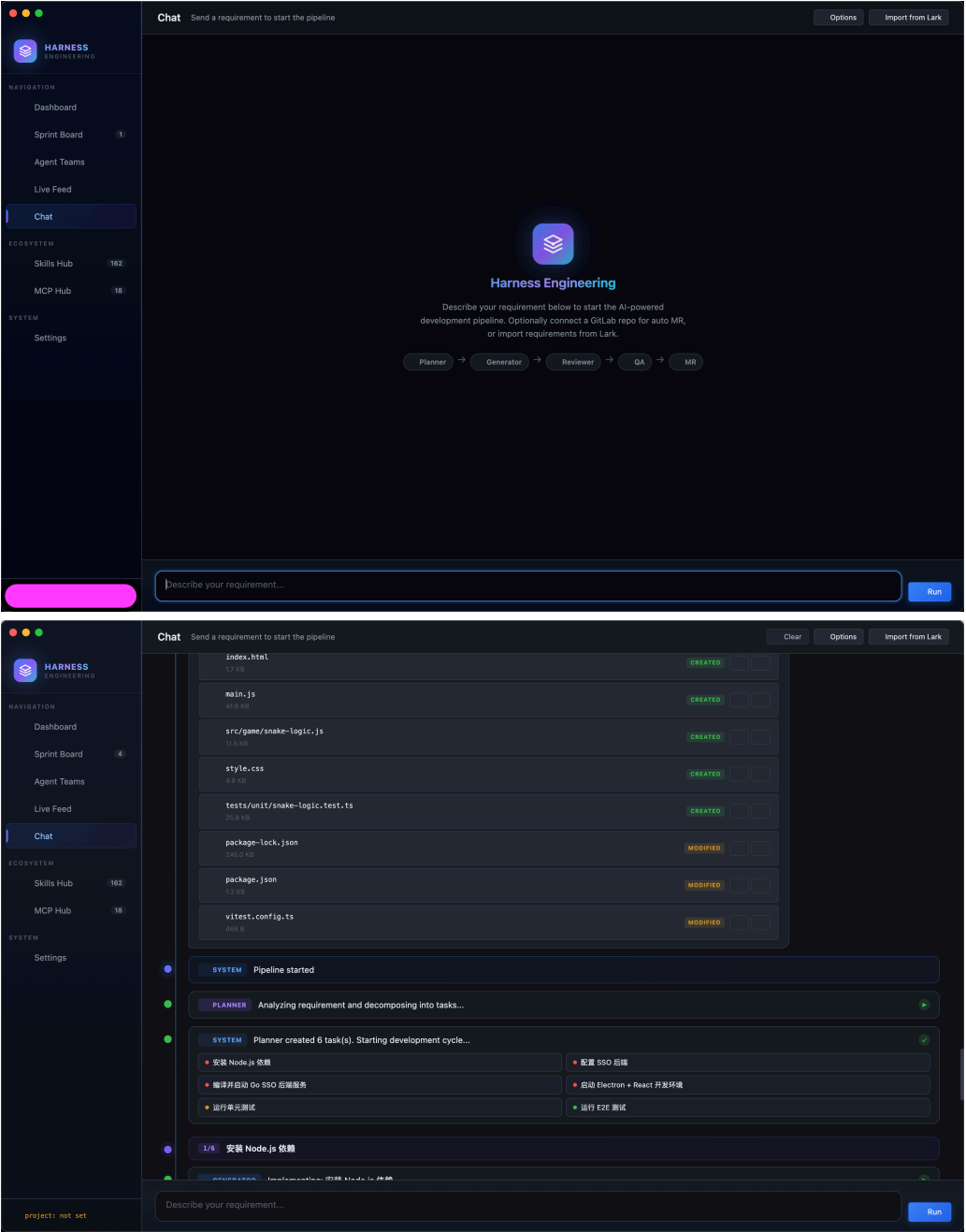
核心能力：

- 大规模事件流：截图中显示 283 events，这是一次完整 Pipeline 执行产生的全部事件。每个事件包含时间戳、智能体名称、事件内容，色彩编码区分不同来源（绿色=agent，青色=pipeline，黄色=system）。
- 六类过滤器：顶部提供 all agent system task_error / stdout 按钮，允许一键筛选关注的事件类型。排查问题时点击 error 即可快速定位失败节点。
- CMD 命令行透视：每个智能体启动时的完整 `claude -p` 命令行以斜体高亮显示，包括 `--output-format stream-json`、`--allowedTools Read,Bash,Glob,Grep`、`--max-turns 30`、`--model openai/sonnet` 等关键参数。这让你能完整复现任何智能体的启动配置。
- 完整执行链路：清晰展现了一次完整的任务执行循环：Generator 启动→思考→执行 Bash 命令→完成(17s)→pipeline 触发 Step 3/4: Code Review→system 移动任务 in_progress→review→Code Reviewer 启动→执行 3 次 Bash 命令→完成(27s)→Step 4/4: QA→QA Engineer 验证→完成(25s)→Pipeline complete。

如果你启动一个 6 小时的 Pipeline，你不知道它正在做第 3 步还是已经卡在某个循环里。Live Feed 解决了这个问题：事件的完整时间线让 Pipeline 的每一步都有迹可循。

9.5 Chat — 流水线执行视图与需求入口





Chat 是用户与 Pipeline 交互的主界面。用户在底部输入框描述需求并点击 Run，Planner 自动拆解任务并启动全流水线。执行过程以任务卡片+消息流的双层结构展示。

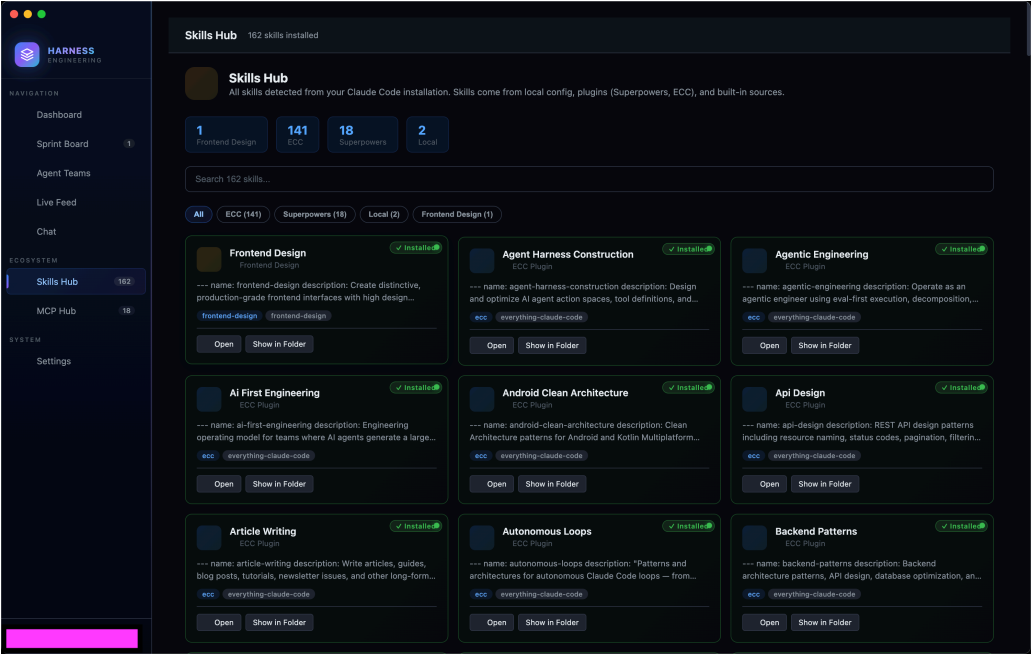
核心能力：

- **任务卡片结构：**每个任务以编号卡片展示（6/7 "Fix identified bugs"、7/7 "Stage and commit change s"），卡片内部按时序展示 Generator–Evaluator 执行链。
- **三角色消息流：**GENERATOR（绿色）显示Implementation complete；CODE REVIEWER（橙色）显示"Review PASSED "；QA ENGINEER（紫色）显示"QA FAILED. Task needs fixes."。颜色编码让用户一笔区分生成、审查、验证三阶段。
- **需求导入：**Import from Lark 按钮可直接从飞书文档拉取需求作为 Pipeline 输入；Options 按钮提供运行参数配置。
- **执行统计：**SYSTEM 消息显示"Pipeline complete! 1/7 tasks done."和"No file changes detected"，后者对应文件快照 Diff 机制。

Chat 的三角色消息流是 Generator–Evaluator 架构的最直观体现。当你看到一个任务卡片内依次出现 GENERATOR → CODE REVIEWER → QA ENGINEER 时，你实际上在观察一个微型 GAN 的运作：生成器产出代码，判别器（Reviewer）评估质量，质检员（QA）做最终裁决。"1/7 tasks done" 的结果看似低效，实则证明了质量门禁的严格性——宁可报告真实结果，不会为了"好看"而降低标准。

9.6 Skills Hub — 技能生态中心





Skills Hub 扫描 Claude Code 安装的所有 Skills 并以商店式卡片布局展示。主要集成了

```
##
https://github.com/obra/superpowers
https://github.com/affaan-m/everything-claude-code
是智能体能力的“军火库”。
##
```

核心能力：

- 分类：Frontend Design 、ECC 、Superpowers 、Local 。ECC（Everything Claude Code）插件提供 141 个专业 Skill，覆盖 Frontend Design、Agent Harness Construction、Agentic Engineering、TDD、API Design、Backend Patterns 等全栈领域。
- 卡片信息架构：每张卡片展示 Skill 名称、来源类型（ECC Plugin / Local）、描述摘要、标签以及安装状态。Open 按钮查看 Skill 完整内容，Show in Folder 打开本地目录。
- 搜索与筛选：顶部搜索框支持关键词搜索 162 个 Skill，分类 Tab 支持按来源筛选，快速定位目标 Skill。

当 Generator 调用 Skill tdd 时，它实际上在读取一套经过精心编写的测试驱动开发最佳实践，而不是仅依赖模型内置知识。这将 Agent 的能力从「模型知道什么」扩展到「团队沉淀了什么」。

9.7 MCP Hub — MCP 服务器管理中心



MCP Hub 读取 `~/.claude/mcp.json` 配置文件，解析并展示所有已安装的 MCP Server，这些 Server 赋予智能体“走出模型”的能力——操作 GitHub、爬取网页、访问数据库、管理部署。

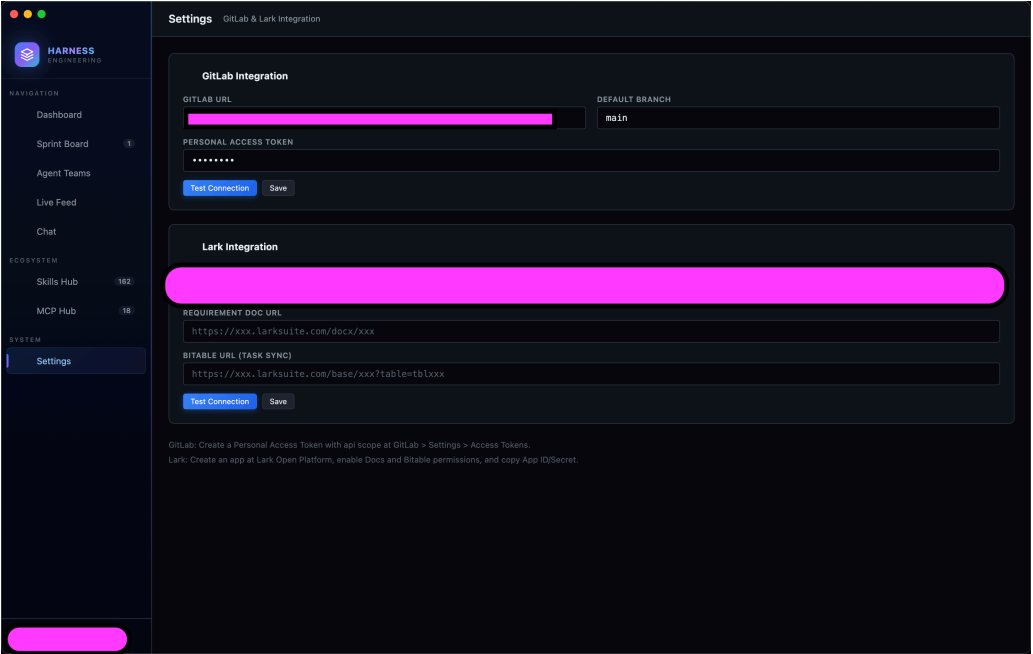
核心能力：

- 分类：Servers 、DevTools 、Other 、AI 、System 。AI 类包含 Memory（跨会话记忆）和 Sequential Thinking（链式推理），这些是增强智能体认知能力的关键服务。
- 卡片信息架构：名称、作用域（Global）、协议类型（npm/remote）、描述、启动命令（如 `npm -y @modelcontextprotocol/server-github`）、所需环境变量（ENV: GITHUB_PERSONAL_ACCESS_TOKEN）、标签和安装状态。Config 按钮支持直接配置环境变量。
- 主流 Server 覆盖：GitHub（PR/Issue操作）、Firecrawl（网页爬取）、Supabase（数据库）、Vercel/Railway/Cloudflare（部署）、Memory（持久记忆）、Sequential Thinking（推理增强）。

MCP Hub 与 Skills Hub 共同构成了第六章描述的「插件生态」。如果说 Skills 是智能体的「知识库」，那么 MCP 就是智能体的「手脚」——它们让 Agent 可以真正操作外部系统。

9.8 Settings — 系统认证集成配置





Settings 是 Harness Engineering 的「外联中枢」，让 Pipeline 不再是封闭的工具，而是融入团队现有工作流的一环。

核心能力：

- **连接验证：**两个模块均提供 Test Connection 按钮，实时验证配置有效性，避免"配完才发现连不上"的问题。
Settings 体现了 Harness Engineering 的「外联」哲学——一个独立的多智能体编排器只是工具，但当它与 GitLab 和 Lark 集成后，就融入了团队的整个工作流。需求从飞书文档来，代码提交到 GitLab 去，任务状态双向同步——这才是生产级工具的样子。

结语：Harness 企业Agent落地困境

claude code和codex的vibe coding能力非常强大，日常开发工作中基本上不需要手写代码，但是企业的AI native研发全流程的工作流还没有迎来“工业革命”，不是AI的能力不够，而是企业上下文环境复杂，不像claude code的codebase，天然是一个高度结构化、系统化的上下文，Agent 的效能高度依赖于对企业内部私有数据的理解。然而，企业数据往往分散在不同的遗留系统和孤岛中，如何低成本、高安全性地为 Agent 提供实时且准确的业务上下文（Context），是技术落地中的巨大瓶颈。

在 Harness Engineering 的实践中，我们学到了几个深刻的教训：

- 架构比模型更重要。模型没法自动解决多智能体协作的核心挑战——上下文管理、质量控制和状态一致性仍然需要精心设计的工程方案。
- 可观测性是信任的基础。当你无法看到 AI 在做什么时，你就无法信任它的输出。Stream-JSON 和实时事件追踪不是锦上添花——它们是生产级 AI 系统的必需品。=
- 约束产生质量。Generator-Evaluator 分离、VERDICT 协议、Sprint 合约——这些约束机制不是在限制 AI 的能力，而是在引导它产生更可靠的输出。
- 持久化不是可选的。任何运行超过 5 分钟的 AI 任务都需要状态持久化。三层保存策略（事件驱动 + 定时 + 退出保存）是我们在多次崩溃后总结出的最小可行方案。

Harness Engineering 的核心理念可以归结为一句话：

“ 无论 AI 多么强大，在生产环境中，人类工程师永远需要知道——系统在做什么，为什么这样做，以及如何在必要时接管控制。这才是 Harness Engineering 的终极命题 ”

原创声明：本文系作者授权腾讯云开发者社区发表，未经许可，不得转载。
如有侵权，请联系 cloudcommunity@tencent.com 删除。



评论

期待你的精彩评论

0/300

发表

8 条评论

热度 最新

- 用户11829451

28天前

大佬 这个平台能开源么

21举报
- 用户10397887

14天前

大佬，能开源吗，学习学习高阶知识

回复 点赞 举报
- 用户3629977

15天前

大佬开源吗

回复 点赞 举报
- 用户1138553

20天前

很厉害的文章

回复 点赞 举报
- 用户4909567

25天前

大佬这个有开源地址吗

回复 点赞 举报
- 用户9781840

27天前

大佬 这个平台能开源么

回复 点赞 举报

推荐阅读

编辑精选文章

换一批

Java与Go差别在哪，谁要被时代...	11650	MCP协议详解：一文读懂跨时代的...	29553
一文掌握 MCP 上下文协议：从理...	10966	LLM 效果不好？可能是 Prompt ...	5839
鹅厂写码13年，我总结的程序员高...	11067	进程，线程，协程 - 你了解多少？	8356

Harness 设计如何支撑Claude长时运行应用开发



设计 系统 开发 模型 前端设计

Harness（脚手架 / 运行编排系统）是 agentic coding（智能体式编程）前沿性能的关键。Anthropic 在这篇文章里展示了：如何通过更精细的 harness 设计，把 Claude 在前端设计和长...

山行AI 2026/04/09 121 0



ClaudeCode–Harness Engineering驾驭者工程的最佳实践者

安全 测试 工具 设计 最佳实践

今天继续跟着我和Claude学习Harness Engineering驾驭者工程。这个我在前面就谈到了是AI编程工程化的一个重点，核心是让AI软件工程更加安全，可控，可靠的运行。



人月聊IT 2026/04/13 1K 0

【译】Harness——用于长时运行应用的智能体框架设计

设计 编辑器 工作 框架 模型

原文： <https://www.anthropic.com/engineering/harness-design-long-running-apps> 作者：Prithvi Rajasekaran，我们的实验室团队成员。

深蓝studyzy 2026/03/28 658 0

Harness Engineering：AI 原生软件开发的未来范式与职业指南

工作 软件开发 设计 产品 工程师

2026年初，OpenAI 发布了一篇名为《Harness Engineering: leveraging Codex in an agent-first world》的技术博客，揭示了一个令人震惊的事实：他们的一个团队用 3 名工程师 + Codex 智能体，在 5 个月内交付了 100 万行代码 的产品，而...

帐篷Li–AIOT–CPS–Token中转 2026/04/02 2.9K 0

6.7% 到 68.3%：Harness Engineering 六大支柱如何重塑 AI Agent 开发

agent

同一个模型，只是换了文件编辑接口的调用方式，编码基准分数从 6.7% 直接跳到 68.3%。模型没变，上下文没变，连 prompt 都没变——变的只是模型外围那套让它和世界交互的系统。

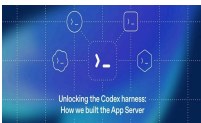


运维有术 2026/04/27 99 0

AI 编程质量不够好，问题可能不在模型，在 Harness

工作 架构 模型 编程 工具

用同一个模型，只改外面那层"壳"，编程基准成功率从 42% 跳到 78%。这是国外 Nate B Jones 今年做的研究，变量只有一个：Harness。



Immerse 2026/04/02 242 0

一文读懂Harness Engineering：从14篇工程文章中，寻找那个让AI不再离经叛道的壳

人工智能

今年三月，LangChain 发布了一篇题为《The Anatomy of an Agent Harness》的实证文章，彻底点燃了所有人的焦虑与狂热。他们在这份报告里引用了一个实验数据对比。仅仅是给同一个大语言模型换上一套更精巧的 Harness 架构，它在...

小腾资讯君 2026/04/03 1.4K 0

爆火的 OpenClaw 和 Harness 什么关系？

设计 系统 游戏 工具 模型

如果你研究过 OpenClaw 的架构，会发现它本质上是一个 Gateway-First 的项目。



臻成AI大模型 2026/04/21 397 0

28个 Agent + 8种 执行模式：把 Claude Code 变成 AI 军团指挥部



测试工具重构agentcode

但用着用着，我发现一个问题：Claude Code 虽然强，可它就像一个超人，什么都能干，但得你一个任务一个任务派。重构完 API，再去写测试，再搞文档，全得排队。

运维有术2026/04/011.3K0



刚水完 OpenClaw，又爆了个 Harness

管理模型设计数据工具

说实话，不知道大家是否同感：AI近两年迭代是真喵快，已经不再是AI一年，人间十年了，是AI一天，人间十年啊。

一臻数据2026/04/154370



AI 不是在抢我的工作：Harness 正在重构软件工程 | 让 Agent 完成任何复杂任务

工作模型软件工程重构agent

OpenAI 和 Anthropic 不约而同提出了「Harness Engineering」（基础设施与脚手架），即驭疆工程。

技术人生黄勇2026/03/301.3K0



从Vibe Coding到Harness Engineering的实践与思考

coding对象架构实践系统

上一篇文章《Multi-Agent系统Harness Engineering架构的思考与实践》我们从 agent 与 MAS 的技术脉络、单 agent 到多 agent 的工程化边界、协调拓扑选择，结合multi-agent项目实践...

小陡坡香菜2026/04/134590

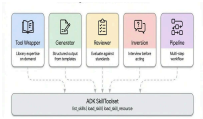


Google 官方：5 种 Agent Skill 设计模式

agentgoogle工具 workflow设计模式

通过研究整个生态系统中技能的构建方式从 Anthropic 的存储库到 Vercel 和 Google 的内部指南。

陈宇明2026/04/104550



Claude Code 居然有 117 个功能，而你可能只用了 3 个

腾讯云AI编程达人秀

大家好，我是程序员太阳鸟，主要介绍副业和 AI 合集，在这里整理了一份开源的 AI 学习资料。

用户31209802026/04/045250

Anthropic 的 Harness 启示：当 AI Agent 开始长跑，架构才是真正的天花板

架构模型设计agent测试

• Anthropic 与五角大楼之争：法院裁定支持 Anthropic，颁布临时禁令阻止五角大楼将其列入供应链风险黑名单，AI 公司与政府机构的博弈进入新阶段。

陆业聪2026/04/023780



2026 年，AI 编程 Agent 的真正分水岭——Harness 详解

agent编程工具模型系统

GPT-5.4 来了，Opus 4.6 来了，Gemini 3.1 来了，Grok 4 也来了

AI学习的老章2026/04/133.2K0



Harness Engineering 给我的启发：用 AI 做 Android 需求，怎么不翻车

模型	android	工具	工作	开发				
• Anthropic 发布多 Agent 协作研究系统，支持并行子任务分发与跨 Agent 结果汇总，适用于长文研究场景，Agent 协同正成为新范式。								
陆业聪	2026/04/02	217	0					

万字深研 | Harness 工程实践：指令遵从率 20%，Hook 执行率 100%

工具	路由	实践	agent	hook				
最近在研究 harness ， AI 不是在抢我的工作： Harness 正在重构软件工程 让 Agent 完成任何复杂任务。								
技术人生黄勇	2026/04/24	129	0					

Skills 深度解析：从原理到实践

人工智能								
本文参考 Anthropic 官方发布的 《The Complete Guide to Building Skills for Claude》，结合工程博客、社区实践和企业案例，深入解析 2025 年 10 月推出、12 月正式开放为行业标准的...								
小陡坡香菜	2026/02/13	2.5K	0					

5 个月 100 万行代码，0 行人工编写——解读AI驾驭工程Harness Engineering

系统	测试	工具	模型	实践				
Planner（规划者）→ Generator（生成者）→ Evaluator（评估者）								
九章智库	2026/04/17	339	0					

社区	活动	圈层	关于	册
技术文章	自媒体同步曝光计划	腾讯云最具价值专家	社区规范	
技术问答	邀请作者入驻	腾讯云架构师技术同盟	免责声明	
技术沙龙	自荐上首页	腾讯云创作之星	联系我们	
技术视频	技术竞赛	腾讯云TDP	友情链接	
学习中心			MCP广场开源版权声明	
技术百科				扫
技术专区				等

热门产品	域名注册 云存储	云服务器 视频直播	区块链服务	消息队列	网络加速	云数据库	
热门推荐	人脸识别 SSL 证书	腾讯会议 语音识别	企业云	CDN加速	视频通话	图像分析	MySQL 数
更多推荐	数据安全 网站监控	负载均衡 数据迁移	短信	文字识别	云点播	大数据	小程序开发